AMRITSAR GROUP OF COLLEGES

Synopsis

On

STUDYFORGE_AI

Subject Name: Major Project Lab (AGCS-21704)

Bachelor Of Technology

In

COMPUTER ENGINEERING

Batch (2023 - 2027)



Submitted To

Er. Ajay Sharma

Associate Professor

Department of CSE

Submitted By

Akshita (2334136)

Aasveer Singh (2334128)

Aryan Rampal (2334150)

Manik Jasrotia (2334191)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDEX

S. No.	Contents	Page No.
1.	INTRODUCTION	3
2.	PROBLEM STATEMENT	4
3.	OBJECTIVES	5
4.	PROPOSED SYSTEM / METHODOLOGY	6
5.	SYSTEM DESIGN	7
6.	TECHNOLOGIES USED	8
7.	HARDWARE & SOFTWARE REQUIREMENTS	9
8.	EXPECTED OUTCOME / DELIVERABLES	10
9.	WORK PLAN / TIMELINE	11
10.	REFERENCES	12

INTRODUCTION

- **Overview of the Project:** StudyForge AI is a full-stack, AI-assisted study workspace that transforms uploaded PDF learning material into cited answers, quiz practice, flashcards, study plans, and visible learning momentum. It combines a Next.js web application with a FastAPI backend, secure authentication, relational persistence, PDF extraction, and Docker-based deployment.
- **Brief Context and Background:** Students often store notes, search for explanations, practise questions, and manage deadlines using separate tools. This breaks the learning workflow and makes it difficult to connect the original study material with the final answer or assessment. StudyForge AI unifies these activities in one personal workspace.
- **Motivation for Choosing the Project:** The project is motivated by the need for trustworthy and organised AI support for students. Rather than providing unsupported generic responses, the platform extracts page-wise content from uploaded PDFs and retrieves relevant source chunks for a cited learning experience.
- **Existing Solutions & Project Differentiation:** ChatPDF and PDF.ai allow document chat but generally do not provide a combined study-plan and progress workflow. Notion and Trello manage tasks but do not understand learning material. General AI chatbots may answer doubts but lack persistent document citations. StudyForge AI differentiates itself by combining PDF-grounded answers, quizzes, flashcards, task management, XP/streak tracking, and a responsive academic interface.

PROBLEM STATEMENT

- **Problem Definition:** To design and develop a secure AI-assisted study platform that converts student-uploaded PDF notes into traceable answers, self-assessment material, and personalised study tasks while maintaining a persistent record of learning progress.
- **Importance:** Students need more than file storage or a generic chatbot: they need to know what to study, practise against their own material, verify the source of answers, and track unfinished work. Solving this problem improves focus, reduces time spent searching through notes, and makes revision more structured.

OBJECTIVES

- **Primary Objectives:**
- Build secure JWT-based registration, login, and protected learning workflows.
- Enable PDF upload and extract page-wise text using PyMuPDF.
- Provide cited, retrieval-based answers using the most relevant extracted source chunks.
- Generate and submit quizzes, record scores, and maintain XP/streak progress.
- Create, update, and complete study-plan tasks from one dashboard.
- **Secondary Objectives:**
- Deliver a responsive modern interface using Next.js 15, React 19, TypeScript, and Tailwind CSS.
- Support a configurable OpenAI-compatible LLM integration while retaining deterministic mock-AI mode for safe demonstrations.
- Package the system with Docker Compose and PostgreSQL/pgvector for scalable deployment.

PROPOSED SYSTEM / METHODOLOGY

- **Proposed Approach:** StudyForge AI uses a modular API-first architecture. The Next.js frontend handles student interaction, while FastAPI provides secure REST APIs for authentication, PDF library management, chat, quiz generation, progress, and study plans. SQLAlchemy maps Python models to SQLite during local development and PostgreSQL with pgvector in Docker deployment.
- **Detailed Workflow:**
- **Authentication:** A student signs up or logs in; JWT tokens protect account-specific pages and APIs.
- **PDF Ingestion:** The student uploads a subject PDF. PyMuPDF extracts the text page by page, and the backend stores document metadata and source chunks.
- **Grounded Retrieval:** For a question, the service compares query terms with stored chunks and selects the most relevant evidence.
- **Learning Activities:** The workspace returns cited answers; quiz and flashcard features help students practise from the same material.
- **Progress Tracking:** Quiz submissions update score data, XP and streaks; study-plan tasks store due dates, estimated minutes, and completion state.
- **Deployment:** Docker Compose runs PostgreSQL/pgvector, FastAPI, and Next.js as isolated services.

SYSTEM DESIGN

- **Database Schema (PostgreSQL / SQLite):** The implemented data model contains USERS, DOCUMENTS, CHUNKS, QUIZ_ATTEMPTS, and STUDY_TASKS. A user owns documents and tasks. Each document contains multiple page-wise chunks that are used for cited retrieval. Quiz attempts record student performance against a document.

ER Diagram:

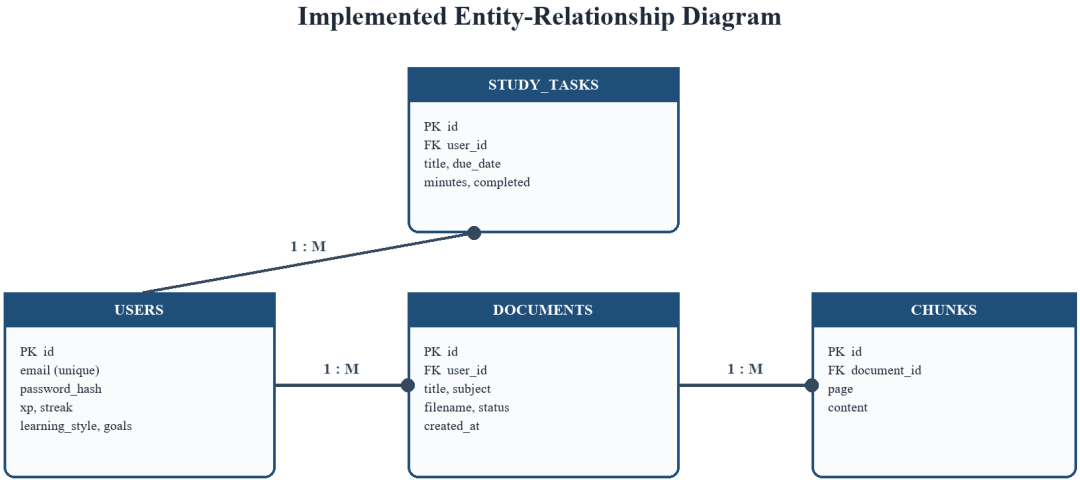


Figure 1: Entity-Relationship Diagram for StudyForge AI

System Architecture Diagram (Conceptual):

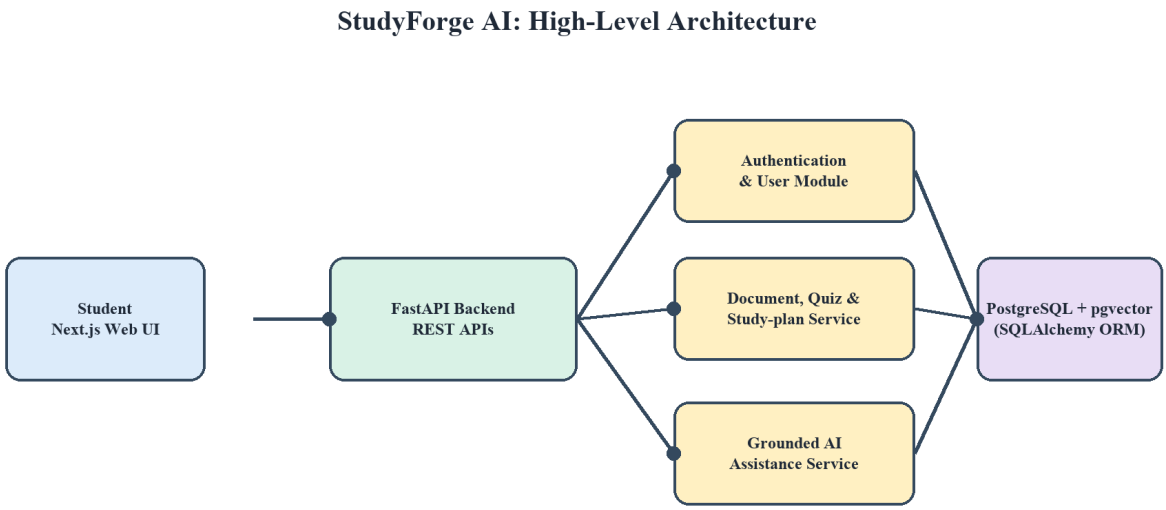


Figure 2: High-level architecture of StudyForge AI

TECHNOLOGIES USED

- **The project is built on a modern full-stack, cloud-ready technology stack:**
- Frontend- Next.js 15 (App Router), React 19, TypeScript, Tailwind CSS, Framer Motion and Lucide React for a responsive, dark-first study interface.
- Backend- Python 3, FastAPI, Uvicorn, SQLAlchemy, Pydantic Settings, python-jose and passlib/bcrypt for high-performance APIs and secure authentication.
- PDF and AI Layer- PyMuPDF for page-wise extraction; an OpenAI-compatible SDK interface for production LLM workflows; safe deterministic mock-AI mode for the demonstration.
- Database- PostgreSQL with pgvector for containerised deployment and vector-search readiness; SQLite for lightweight local development.
- DevOps- Docker and Docker Compose for reproducible multi-service deployment; pytest and httpx for automated backend/API testing.
- Cloud-Native Direction- AWS, GCP, or Azure can host the API, database, object storage, monitoring, and future scalable services.

HARDWARE & SOFTWARE REQUIREMENTS

• **Minimum Hardware Requirements:**

• **Processor:** Intel Core i3 / AMD equivalent or higher.

• **RAM:** 4 GB minimum; 8 GB recommended for Docker, browser, and development tools.

• **Storage:** 2 GB free space for code, database, packages, and uploads.

• **Internet:** Required for dependencies, optional LLM API access, and cloud deployment.

• **Software Requirements:**

• **OS:** Windows 10/11, Linux, or macOS.

• Python 3.10+ with pip and virtual environment; Node.js 20+ with npm.

• VS Code, Git, Docker Desktop, Postman/Insomnia, and a modern browser.

• FastAPI, Uvicorn, SQLAlchemy, PostgreSQL/SQLite, Next.js, and Docker Compose.

EXPECTED OUTCOME / DELIVERABLES

- **Expected Outcome:** A fully functional AI-assisted study workspace that students can access through a responsive web interface. Users can securely create an account, upload PDF notes, ask cited questions, practise with quizzes and flashcards, plan study tasks, and observe learning momentum through XP, streaks, and progress data.
- **Key Deliverables:**
- Complete Next.js frontend and FastAPI backend source code.
- JWT authentication, document library, PDF chunking, cited chat, quizzes, flashcards, study plans, and progress features.
- PostgreSQL/SQLite database schema with SQLAlchemy models.
- Docker Compose configuration for one-command local deployment.
- ER diagram, architecture diagram, test suite, project report, and live demonstration.

WORK PLAN / TIMELINE

Phase	Weeks	Milestone
Requirement Analysis	1-2	Study workflow, use cases, scope and feasibility
Design	3-4	Architecture, ER diagram, API contracts and UI design
Core Development	5-7	Authentication, database, upload and PDF extraction
Learning Modules	8-10	Cited chat, quizzes, flashcards, tasks and progress
AI and Integration	11-12	LLM service integration and protected client workflows
Testing and Deployment	13-16	API testing, Docker deployment, report and final demo

REFERENCES

• FastAPI Documentation. https://fastapi.tiangolo.com/

• Next.js Documentation. https://nextjs.org/docs

• SQLAlchemy Documentation. https://docs.sqlalchemy.org/

• PyMuPDF Documentation. https://pymupdf.readthedocs.io/

• PostgreSQL Documentation. https://www.postgresql.org/docs/

• OpenAI API Documentation. https://platform.openai.com/docs/

• Docker Documentation. https://docs.docker.com/